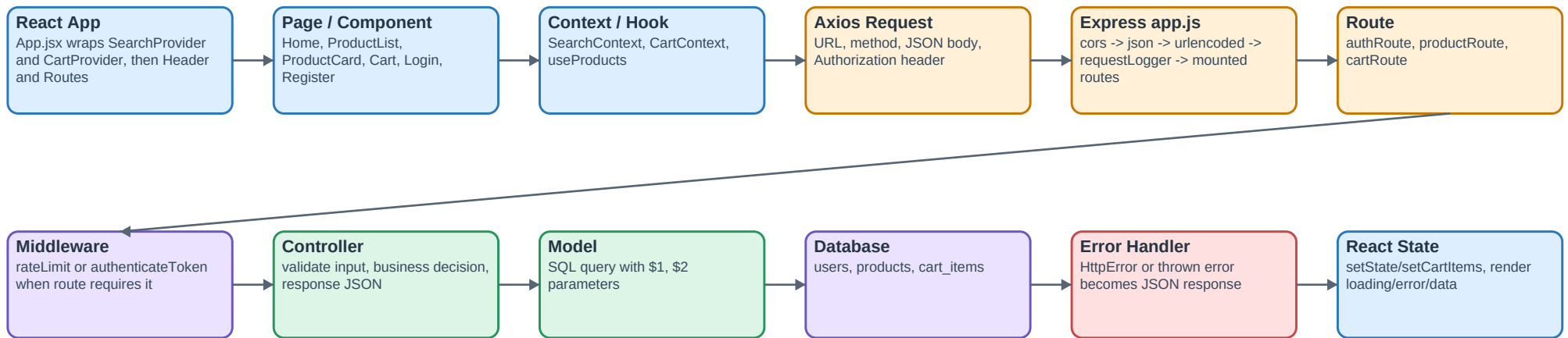


Deep End-to-End Architecture

Request lifecycle from React click to PostgreSQL and back



Core rule

- Frontend only knows the JWT string in localStorage. It does not know the database user id directly.
- Backend verifies the JWT in authenticateToken. After verification, req.user contains the decoded payload.
- Cart controller reads Number(req.user.sub). That value becomes user_id in cart_items.
- Every controller either sends a success JSON response or throws HttpError; errorHandler formats failures.

API Routes and JSON Response Map

All endpoints with request body, success JSON, and common failures

Endpoint	Input	Success JSON	Common errors
POST /auth/register	{ username, email, password }	201 { success:true, message, user, token }	409 user exists, 422 validation
POST /auth/login	{ email, password }	200 { message:'Login successful', token }	401 user/password, 429 rate limit, 422
GET /products	none	200 { message, count, products:[...] }	500 database/server
GET /products/filter	query: name/category/brand/price/stock/rating/inStock	200 { message, count, products:[...] }	404 no matches, 422 invalid query
GET /products/:id	path id	200 { message, products: product }	404 not found, 422 invalid id
POST /products	Bearer token + product JSON	201 { message, product }	401 auth, 422 body
PUT /products/:id	Bearer token + full product	200 { message, product }	401 auth, 404, 422
PATCH /products/:id	Bearer token + partial product	200 { message, product }	401 auth, 404, 422
DELETE /products/:id	Bearer token + path id	200 { 'Deleted Product': product }	401 auth, 404, 422
DELETE /products	Bearer token	204 No Content	401 auth
GET /cart	Bearer token	200 { items:[products with quantity] }	401 missing/expired/invalid token
POST /cart	Bearer token + { productId }	200 { items:[...] }	401 auth, 404 product, 422 body
DELETE /cart/:productId	Bearer token + path productId	200 { items:[...] }	401 auth, 422 id

Product object shape

```
{
  id, name, price, stock, instock,
  brand, category, rating, quantity?
}
```

quantity appears in cart responses because cartModel joins products with cart_items.

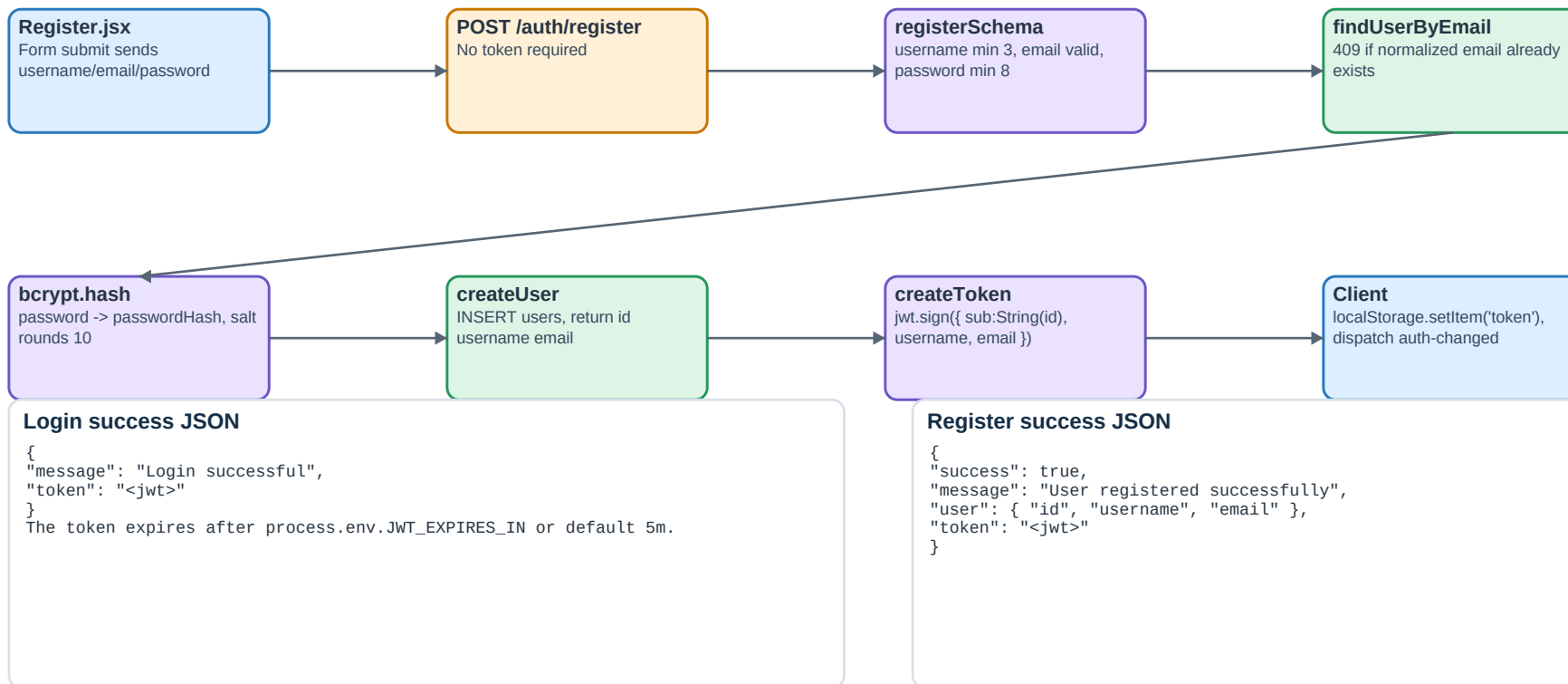
Central error JSON shape

```
{
  "success": false,
  "message": "human readable error"
}
```

Thrown HttpError/status errors are normalized by backend/middlewares/errorHandler.js.

Authentication Workflow - Deep View

Register and login both finish by creating the same kind of JWT

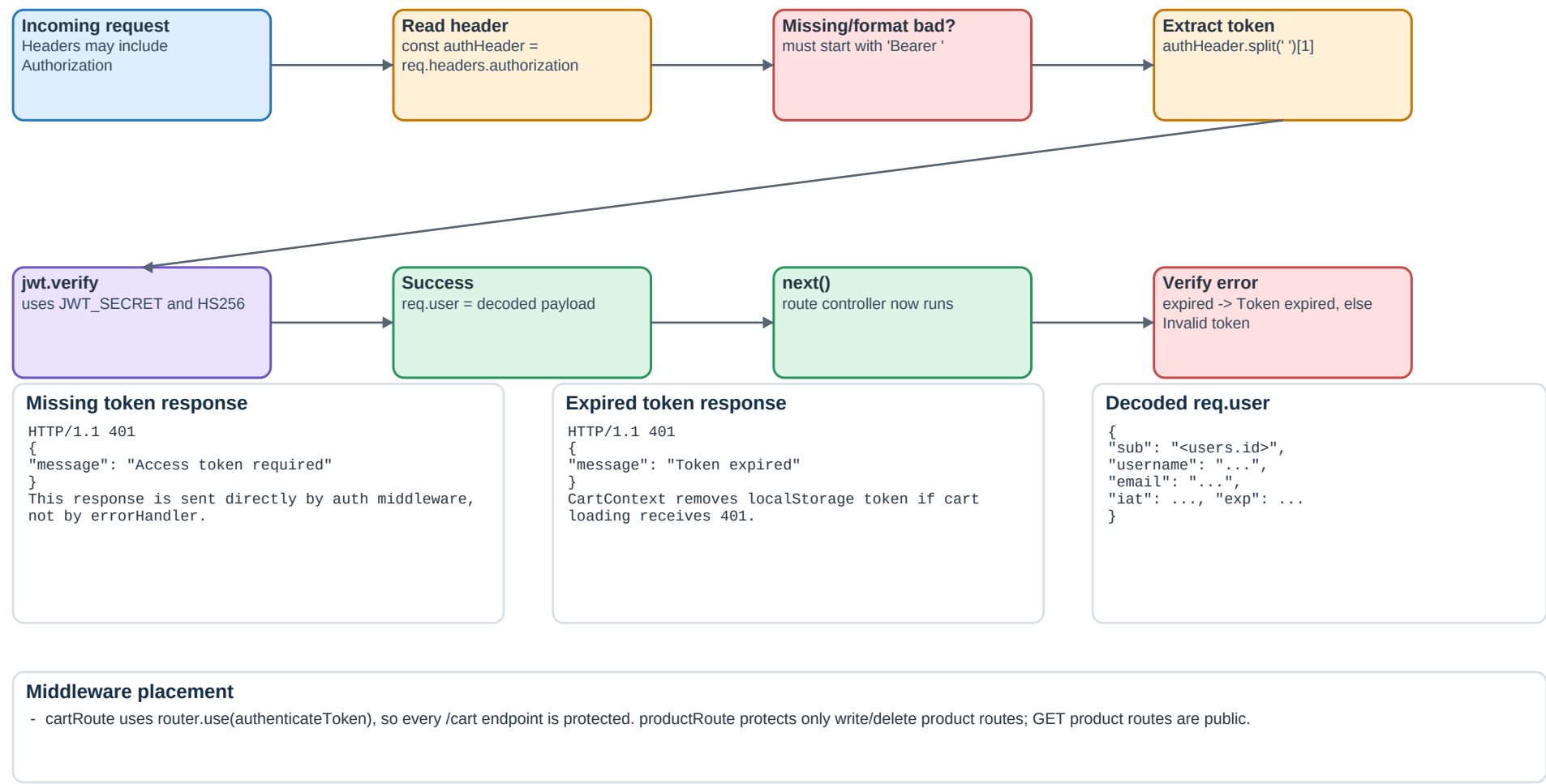


AuthRoute is frontend-only protection

- AuthRoute only checks localStorage.token before showing /cart or /dashboard. Real API security is authenticateToken on the backend. A fake token can pass AuthRoute, but backend rejects it with 401.

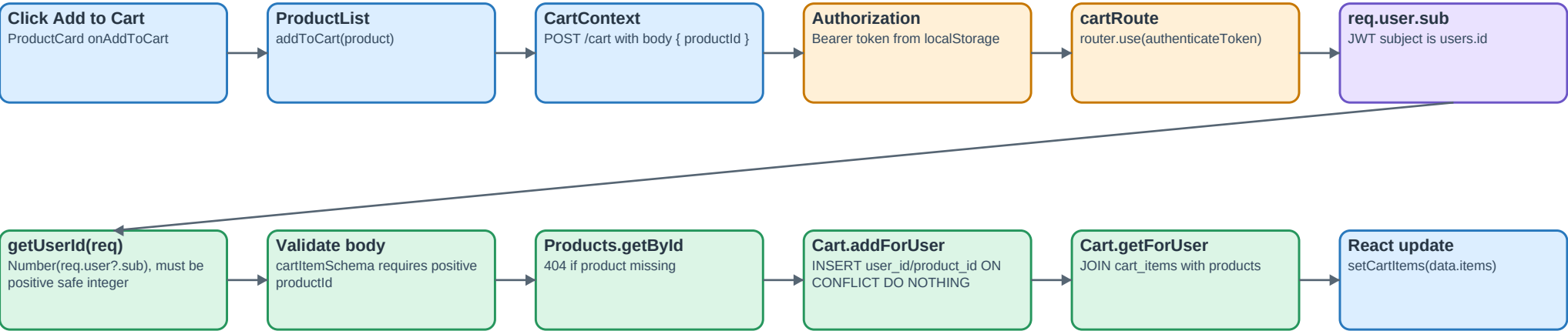
authenticateToken Middleware

Server-side JWT gate used by cart routes and protected product write routes



Cart Workflow - Add, Load, Remove

The cart user is always derived from req.user.sub after JWT verification



POST /cart request

```
POST /cart
Authorization: Bearer <jwt>
Content-Type: application/json

{ "productId": 3 }
```

POST /cart success

```
HTTP/1.1 200
{
  "items": [
    { product fields..., "quantity": 1 }
  ]
}
```

DELETE /cart/:productId

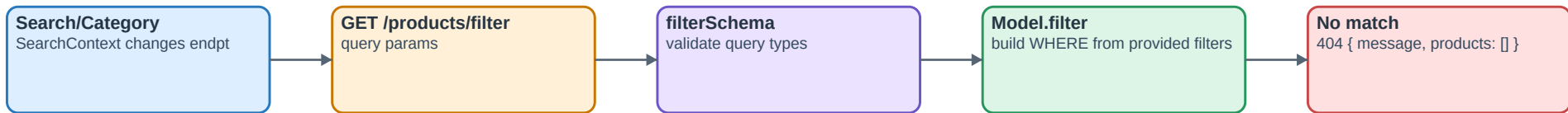
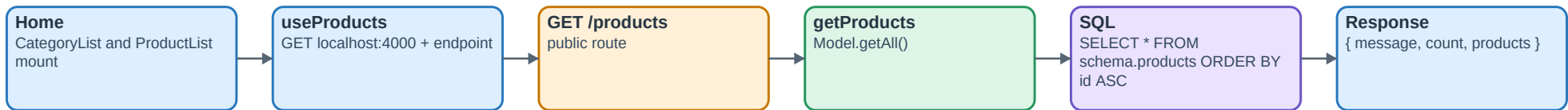
- Same auth path. Controller validates `req.params.productId`, calls `Cart.removeForUser(userId, productId)`, then returns `{ items }` from `Cart.getForUser(userId)`.

Cart table behavior

- `cart_items` primary key is `(user_id, product_id)`, so each product appears once per user. `addForUser` uses `ON CONFLICT DO NOTHING`; clicking add again does not increase quantity in the current code.

Products, Search, Filter, and Product Write Flow

Public reads, protected writes, Joi validation, SQL model calls



Product filter example

```
GET /products/filter?name=bag

200 {
  "message": "Product data fetched",
  "count": 2,
  "products": [...]
}
```

Protected write request

```
PATCH /products/5
Authorization: Bearer <jwt>

{ "price": 999, "stock": 4 }
```

Requires authenticateToken first.

Write validation

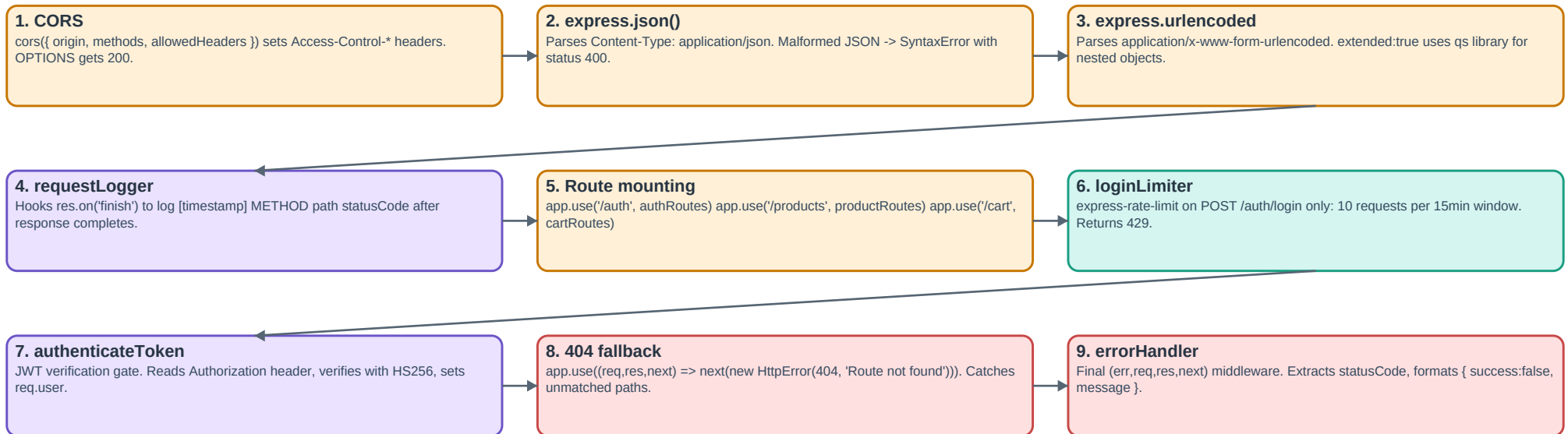
- POST/PUT use productSchema. PATCH uses patchSchema.min(1). id params use idSchema. Validation failure throws HttpError(422).

Important naming detail

- Database column is instock, while request schema uses inStock. Controller/model translate inStock to instock during create/update.

Middleware Pipeline Deep Dive

Every middleware in execution order with what it does to req/res



Middleware order matters

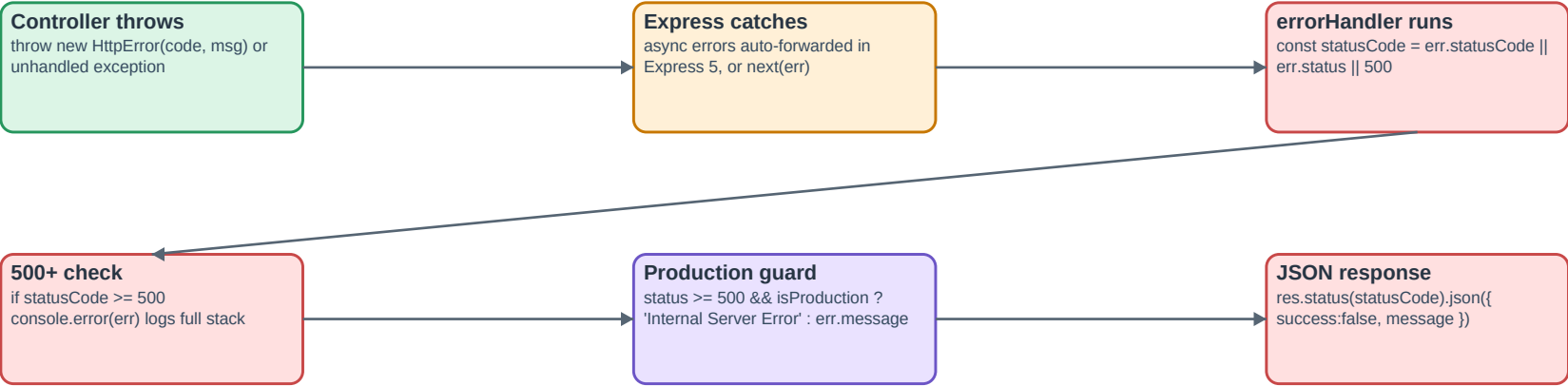
- CORS must come first so preflight OPTIONS requests get proper headers.
- express.json() must run before any route that reads req.body.
- requestLogger hooks res.finish early so it captures ALL responses including errors.
- errorHandler MUST be last (4 args) - Express only routes errors to it if it has 4 parameters.

Rate limiter config (authRoute.js)

```
windowMs: 15 * 60 * 1000 // 15 minutes
limit: 10 // max 10 per window
standardHeaders: true // RateLimit-* headers
legacyHeaders: false // no X-RateLimit-*
statusCode: 429
// { message: "Too many login attempts, try again later" }
```

Error Handler and HttpError Deep Flow

How every error type becomes a structured JSON response



Code	Type	Trigger	Source
400	Malformed JSON	express.json() parser rejects invalid JSON body	SyntaxError with status 400
401	Auth failed	authenticateToken: missing header, invalid or expired JWT	Direct res.json() from auth.js
401	Bad user sub	getUserId(req) finds non-integer or <= 0 sub in JWT	HttpError thrown in cartController
404	Not found	Product/route not found: getProductById, 404 fallback in app.js	HttpError(404, message)
409	Conflict	registerUser: email already exists in users table	HttpError(409, 'User already registered')
422	Validation	Joi schema.validate fails: abortEarly:false collects all errors	HttpError(422, joined messages)
429	Rate limit	loginLimiter: > 10 POST /auth/login in 15min	express-rate-limit auto-response
500	Server error	Unhandled exception, database connection failure, query error	Production: msg hidden

HttpError utility (utils/httpError.js)

```
function HttpError(statusCode, message) {  
  const error = new Error(message);  
  error.name = 'HttpError';  
  error.statusCode = statusCode;  
  return error;  
}
```

validate utility (utils/validate.js)

```
function validate(schema, data) {  
  const { error, value } = schema.validate(data,  
    { abortEarly: false });  
  if (error) throw new HttpError(422,  
    error.details.map(d => d.message).join(', '));  
  return value;  
}
```

Response JSON Catalog - Every Shape

Exact JSON body your frontend receives for success and failure cases

POST /auth/register 201

```
{
  "success": true,
  "message": "User registered successfully",
  "user": {
    "id": 1,
    "username": "john",
    "email": "john@example.com"
  },
  "token": "eyJhbGciOiJIUzI1NiIs..."
}
```

POST /auth/login 200

```
{
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIs..."
}

// Token payload (decoded):
{ "sub": "1", "username": "john",
  "email": "john@example.com",
  "iat": 1728050000, "exp": 1728050300 }
```

GET /products 200

```
{
  "message": "Product data fetched",
  "count": 25,
  "products": [
    {
      "id": 1,
      "name": "Backpack",
      "price": 2499,
      "stock": 10,
      "instock": true,
      "brand": "Nike",
      "category": "Bags",
      "rating": 4.2
    }, ...
  ]
}
```

GET /cart 200

```
{
  "items": [
    {
      "id": 3,
      "name": "Sneakers",
      "price": 5999,
      "stock": 5,
      "instock": true,
      "brand": "Adidas",
      "category": "Shoes",
      "rating": 4.5,
      "quantity": 1
    }
  ]
}
```

POST /cart 200

```
// Same structure as GET /cart
// Returns full cart after adding item
{
  "items": [
    { ...product, "quantity": 1 },
    { ...product, "quantity": 1 }
  ]
}

// ON CONFLICT DO NOTHING means
// duplicate add is silently ignored.
```

GET /products/filter?name=bag 200

```
{
  "message": "Product data fetched",
  "count": 2,
  "products": [ ... ]
}

// No match returns 404:
{
  "message": "No products matched",
  "products": [],
  "availableCategories": [
    "Bags", "Shoes", ...
  ]
}
```

Error responses (all error types)

```
// 401 Unauthorized (from auth middleware directly)
{ "message": "Access token required" }
{ "message": "Token expired" }
{ "message": "Invalid token" }

// 422 Validation (from errorHandler via HttpError)
{ "success": false, "message": "\"email\" must be valid" }

// 404 Not Found
{ "success": false, "message": "Product not found" }
{ "success": false, "message": "Route /xyz not found" }

// 409 Conflict
{ "success": true, "message": "User already registered" }

// 429 Rate Limited
{ "message": "Too many login attempts, try again later" }
```

Response pattern rules

- Success responses from controllers always include a 'message' field describing what happened.
- Product endpoints use { message, count?, products } pattern.
- Cart endpoints use { items: [...] } pattern with joined product+quantity data.
- Auth endpoints return { token } on success. Register also returns { user }.
- Error responses from errorHandler always have { success: false, message }.
- Auth middleware (401) sends { message } directly, NOT through errorHandler.
- Rate limiter (429) sends its own { message }, also NOT through errorHandler.
- DELETE /products returns 204 No Content with empty body for bulk delete.

Joi Validation Schemas Deep Dive

Every schema rule with data types, constraints, defaults, and when each is triggered

authSchema.js - registerSchema

```
Joi.object({
  username: Joi.string().min(3).required(),
  email: Joi.string().email().required(),
  password: Joi.string().min(8).required(),
})
```

Used by: POST /auth/register -> registerUser
Trigger: validate(Schema.registerSchema, req.body)
Failure: HttpError(422) with joined detail messages

Example error:
"username" length must be at least 3 characters long,
"password" length must be at least 8 characters long

authSchema.js - loginSchema

```
Joi.object({
  username: Joi.string().min(3), // optional
  email: Joi.string().email().required(),
  password: Joi.string().min(8).required(),
})
```

Used by: POST /auth/login -> loginUser
Note: username is optional in login - only email + password are checked against DB.
After validation, controller calls findUserByEmail(email.toLowerCase())

productSchema.js - productSchema (POST/PUT)

```
Joi.object({
  name: Joi.string().min(2).required(),
  category: Joi.string().min(2).required(),
  price: Joi.number().integer().min(100).required(),
  brand: Joi.string().min(2).required(),
  stock: Joi.number().integer().min(0).default(1),
  rating: Joi.number().min(0).max(5).default(1),
  inStock: Joi.boolean().default(false),
})
```

Used by: POST /products, PUT /products/:id
Defaults: stock=1, rating=1, inStock=false

productSchema.js - patchSchema (PATCH)

```
Joi.object({
  name: Joi.string().min(2),
  category: Joi.string().min(2),
  price: Joi.number().integer().min(100),
  brand: Joi.string().min(2),
  stock: Joi.number().integer().min(0),
  rating: Joi.number().min(0).max(5),
  inStock: Joi.boolean(),
}).min(1)
```

At least one field required (.min(1))
Controller merges with existing product values

cartSchema.js

```
cartItemSchema = Joi.object({
  productId: Joi.number().integer()
    .positive().required(),
})
```

```
idSchema = Joi.object({
  id: Joi.number().integer()
    .positive().required(),
})
```

idSchema (products + cart)

```
// Used for all :id path params
Joi.object({
  id: Joi.number().integer()
    .positive().required(),
})
```

```
// Product controller wraps this:
const validId = (params) =>
  validate(Schema.idSchema, params).id;
```

Validation flow summary

- 1. Controller calls validate(schema, data)
- 2. Joi runs .validate() with abortEarly:false
- 3. All errors collected in error.details[]
- 4. Details mapped to d.message and joined
- 5. HttpError(422, joinedMessages) thrown
- 6. errorHandler catches and sends JSON

Full-Stack Authentication Sequence

Complete timeline from user click to database to protected API call

Register -> Token -> Protected Cart Request (full sequence)

- STEP 1: User fills Register form (username, email, password) and clicks Submit
- STEP 2: Register.jsx calls `axios.post('http://localhost:4000/auth/register', { username, email, password })`
- STEP 3: Express routes to `authRoute -> registerUser` controller
- STEP 4: `validate(registerSchema, req.body)` checks: `username >= 3` chars, email valid format, `password >= 8` chars
- STEP 5: `email.toLowerCase()` normalizes the email for case-insensitive lookup
- STEP 6: `findUserByEmail(normalizedEmail)` -> if found, throw `HttpError(409, 'User already registered')`
- STEP 7: `bcrypt.hash(password, 10)` creates a salted hash (10 rounds). Original password is never stored.
- STEP 8: `createUser({ username, email, passwordHash })` -> INSERT INTO users RETURNING id, username, email
- STEP 9: `createToken(user)` -> `jwt.sign({ sub: String(user.id), username, email }, JWT_SECRET, { expiresIn: '5m' })`
- STEP 10: Response 201 { success: true, message, user: { id, username, email }, token: '<jwt>' }
- STEP 11: Client stores `localStorage.setItem('token', token)` and dispatches window event 'auth-changed'
- STEP 12: `CartContext` listens for 'auth-changed', reads new token, fires GET /cart with `Authorization: Bearer <token>`
- STEP 13: `authenticateToken` verifies JWT -> `req.user = { sub: '1', username: 'john', email: 'john@x.com', iat, exp }`
- STEP 14: `getUserId(req)` -> `Number(req.user.sub) = 1` -> `Cart.getForUser(1)` -> returns cart items for user 1

Login -> Token -> Auto-Expiry Handling

- STEP 1: Login.jsx submits `axios.post('http://localhost:4000/auth/login', { email, password })`
- STEP 2: `loginLimiter` checks: has this IP made > 10 login attempts in the last 15 minutes? If yes -> 429.
- STEP 3: `validate(loginSchema, req.body)` checks email format and `password >= 8` chars
- STEP 4: `findUserByEmail(email.toLowerCase())` queries `SELECT * FROM users WHERE email = $1`
- STEP 5: If no user found -> throw `HttpError(401, 'User Not Found')`. Login.jsx redirects to /register.
- STEP 6: `bcrypt.compare(password, user.password)` compares submitted password against stored bcrypt hash
- STEP 7: If hash mismatch -> throw `HttpError(401, 'Invalid email or password')`
- STEP 8: `createToken(user)` generates JWT with same structure as registration
- STEP 9: Response 200 { message: 'Login successful', token }. Client stores and dispatches auth-changed.
- STEP 10: Token expires after `JWT_EXPIRES_IN` (default 5m). `AuthRoute.jsx` also has `setTimeout(5min)` to clear token.
- STEP 11: When expired token is used in cart requests, `authenticateToken` catches `TokenExpiredError` -> 401
- STEP 12: `CartContext.catch()` detects 401 -> `localStorage.removeItem('token')` -> `setToken(null)` -> cart clears

Security boundaries summary

- Frontend `AuthRoute`: UX guard only. Checks if `localStorage` has ANY token string. Does NOT verify validity. A fake token passes `AuthRoute` but fails at backend.
- Backend `authenticateToken`: Real security. Verifies JWT signature (HS256 + `JWT_SECRET`), checks expiry, decodes payload. Only passes if cryptographically valid.
- Database layer: `user_id` comes ONLY from verified JWT sub claim, never from request body. SQL uses parameterized queries (`$1`, `$2`) to prevent injection.

Step-by-Step Auth + Cart Mental Model

Use this page to explain the hardest part clearly

1. Login creates identity proof

- User submits email/password from Login.jsx.
- Backend finds the user by lowercased email.
- bcrypt.compare checks password against stored hash.
- jwt.sign creates token with sub equal to database users.id.
- Client stores only the token, not the user id.

2. Cart request sends proof

- CartContext reads token from localStorage.
- GET /cart, POST /cart, DELETE /cart/:productId include Authorization: Bearer <token>.
- If localStorage has no token, CartProvider clears cart state.
- If server returns 401, CartProvider removes the token.

3. Backend finds auth user

- cartRoute runs authenticateToken before every cart controller.
- jwt.verify checks signature, expiry, and algorithm.
- On success, req.user becomes decoded payload.
- getUserId(req) converts req.user.sub to a number.
- Invalid or missing sub throws HttpError(401).

4. Cart model uses that user id

- getUser(userId): SELECT product rows joined to this user's cart_items.
- addForUser(userId, productId): INSERT row for this user/product pair.
- removeForUser(userId, productId): DELETE only this user's row.
- The API never trusts product/user ids from the frontend for user identity.

Final one-line explanation

- Authentication answers: who is making this request? Cart answers: which products belong to that authenticated user? The link between them is the JWT sub claim. After middleware verifies the token, req.user.sub becomes the trusted database user id used in every cart query.